lecture 9
2-3 trees

Searching with trees

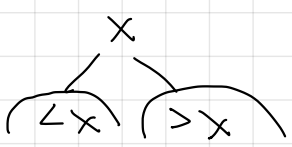
Red Black Trees (std::map) } related to 2-3 trees
B-Trees (databases) }

Goal: Balanced Binary Tree

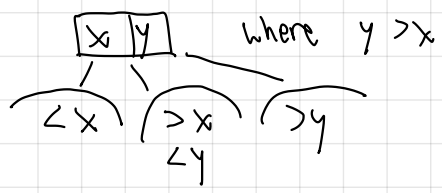
AVL Tree relax "balanced" definition
2-3 Trees relax "binary" definition

2-3 Trees: A perfectly balanced tree where all leaves are on the same level. every node looks like: have all or none of your children

2-node



3-node

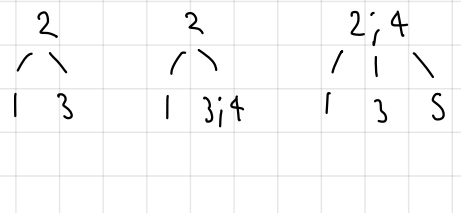

X Y
where Y >X
<X
>X
<Y
>Y

will and many the 2 and 3 nodes

A balanced BST is a 2-node tree

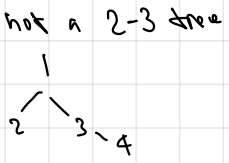
Valid 2-3 trees

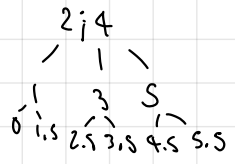
all leaves need to be on the same level



assume no duplicates

not a 2-3 tree





heap could be stored in an array since it is always complete
AVL trees, B trees, 2-3 trees use nodes and pointers

Values 1,2,3,4,5

Valid 2-3 trees



Same as binary search, but at most 3 comparisons

since perfectly balanced

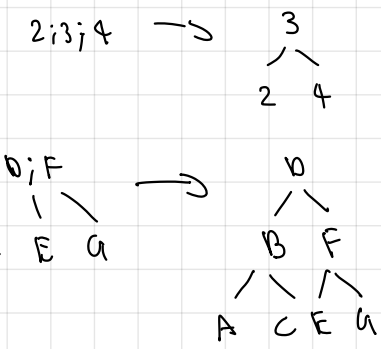
2-3 trees with n nodes has at most O(log(n)) levels

2-3 Tree Insert
- rely on split and absorb

Insertion
- reach leaf and insert
- if 4 node, split it
- fix balance with absorb

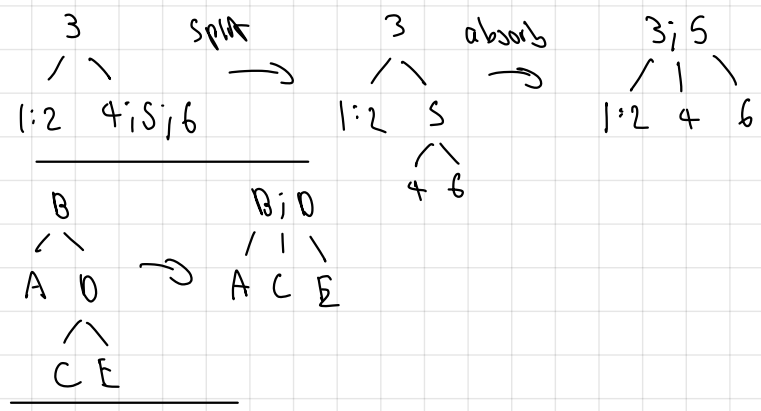
O(log(n))

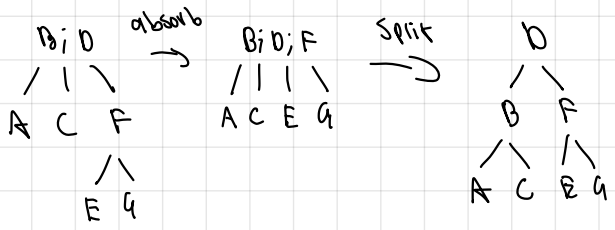
Split
- Given root node 2;3, you need to insert 4
- take middle value and pull it upwards
- recall nodes can have 1 or 2 values



modify structure while maintaining order

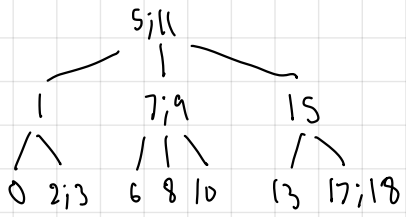
Absorb
- give a 2-node to its parent to fix the balance
- the parent becomes a 3-node



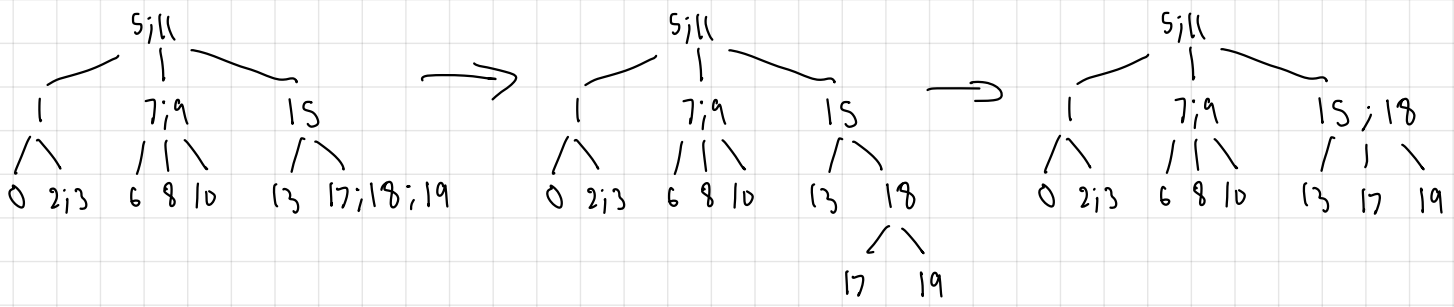



```
def insert(v, root
  if v in root.values:
      return;
  if root is empty:
      replace root with new leaf
  elif root is leaf:
      add v to root.values
  else:
      child ∈ find child for value v
      insert(child, v)
      if child split:
          absorb into root
  if root is a 4-node
      split the root
```

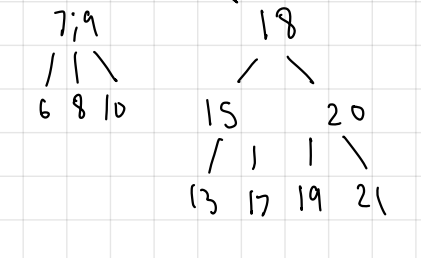
Given:



insert 19

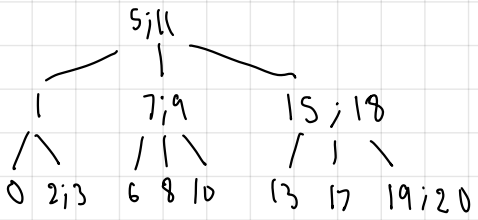


Given:



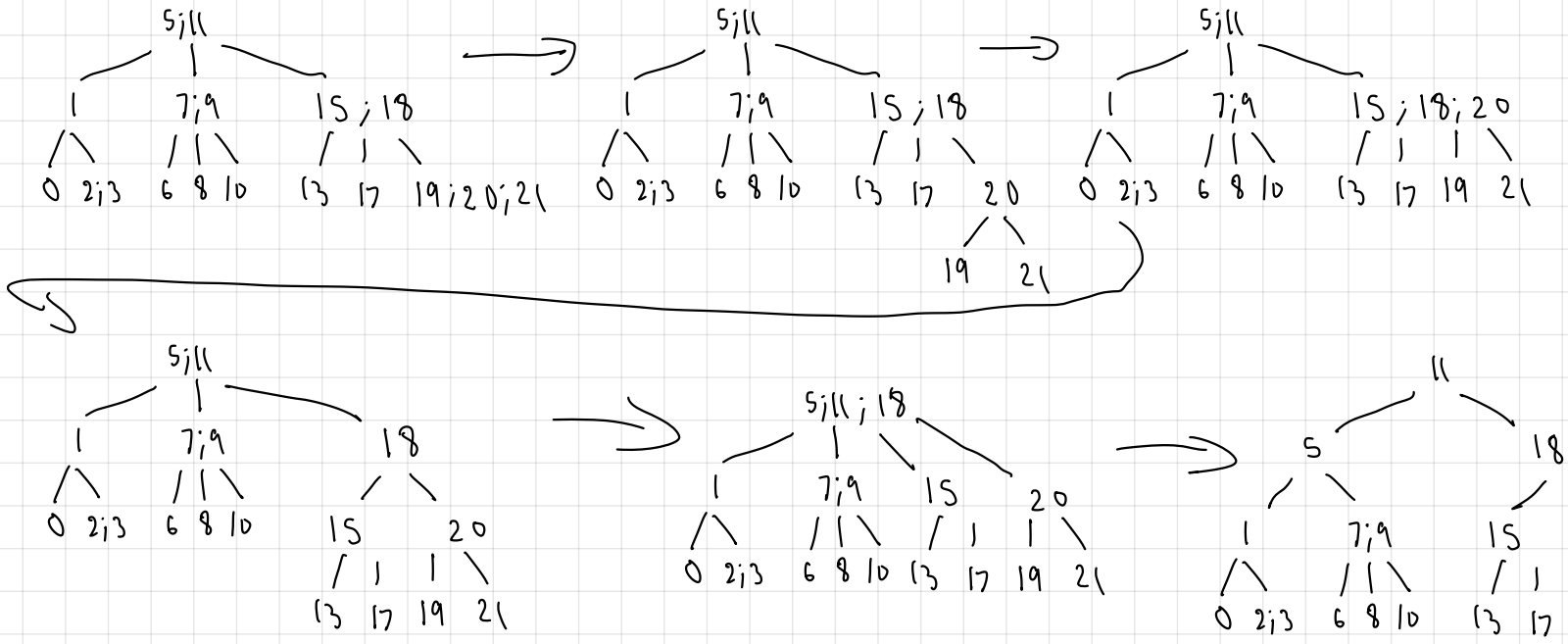
rules
1. If you make a 4-node, split it
2. If your child splits, absorb it

insert 20



splitting the root is the only way to add levels
grow from the root

insert 21



no need to absorb once you reach a 2-node
- the 2-node becomes a 3-node

B-Tree
- pick branching factor K. at most K ways to branch
- self balancing
- all leaf nodes appear at the same level
- keys within a node act as separators that divide the ranges of the child subtrees

every node has at most m children
every internal node except the root has at least $\lceil m/2 \rceil$ children
the root has at least 2 children (unless it is a leaf)
all leaves the same depth

really slow from memory, so use caching

Brainstorm: why a B Tree is better than a BST

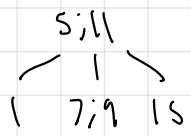
pros:
- with more caching, there are more opportunities to cache adjacent memory
- shorter tree

cons:
- larger if requires more comparisons when searching
- slow insertion, just an array. need to shift everything

Red-Black Tree

represent a 2-3 tree with a binary tree representation by coloring nodes

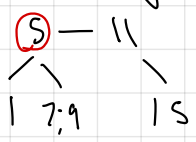
Ex:



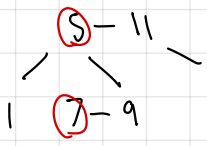
convert 2-3 tree to BST
1. split 3 nodes. color node pushes to left neg

split 5;11

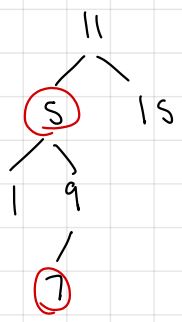
left leaning. middle node goes to middle



split 7;9



2. pick up by a black node



can do all 2-3 operations on a BST

4

lecture 10
amortized time

data structure	worst case for 1 operation	worst case for n operations	amortized time
A	$O(\sqrt{n})$		
B	$O(n)$	$O(n \log(n))$	$\frac{n\log(n)}{n} = O(\log(n))$

Data structure B is more scalable for big values of N

A: $n\sqrt{n}$
B: $n\log(n)$ } $n\log(n)$ is less than $n\sqrt{n}$

There are scenarios where you want a constant running time
- especially when code runs at special intervals

In these cases, data structure A is better since it is consistent

---

new tradeoff: time for 1 operation vs time for n sequence of operations

amortized time: average time for n operations

can have bad individual time, but average good time

$O(A(n))$ amortized time means n operations take $O(n\,A(n))$ total time

---

Growing Array

only 1 operation: appends value to end of array

[1, 2, 3, 4]
push 5

Challenge: you will run out of space and need to copy everything over to new memory, also free old memory

Naive design: array is always full and need to allocate and copy to new data

```
def push(array, val):
    new_data ← [request array, N+1 space]
    for i ∈ 0, 1, ..., array.N-1:
        new_data[i] = array.data[i]
    release array.data
    new_data[array.N] ← val
    array.data = new_data
    array.N += 1
```

Inserting value	1	2	3	4	5	6	7	8	9
n after insertion	1	2	3	4	5	6	7	8	9
size after insertion	1	2	3	4	5	6	7	8	9
time for insertion	$\le C$	$\le 2C$	$\le 3C$	$\le 4C$	$\le 5C$	$\le 6C$	$\le 7C$	$\le 8C$	$\le 9C$

Total: $\le 1C + 2C + 3C + \ldots + n \cdot C = C(1+2+3\ldots+n)$
$= C \cdot \frac{1}{2} n(n+1)$
$= O(n^2)$

amortized: $O(\frac{n^2}{n})$
$= O(n)$

divide total time for n operations by n to get amortized time

average case time: average over input distribution
amortized time: average of number of operations

only for algorithms
only for data structures

Adjustment: double array size when you request for more space

```
def push(arr, val):

    if arr.n == arr.s:
        new_data ← (request 2·(arr.N+1))

        for i ← 0,1,..., arr.N-1:
            new_data[i] ← arr.data[i]

        release old data
        arr.data ← new_data
        arr.S ← 2·(arr.N+1)
    arr.data[arr.N] ← val
    arr.N += 1
```

Inserting value	1	2	3	4	5	6	7	8	9
n after insertion	1	2	3	4	5	6	7	8	9
size after insertion	2	2	6	6	6	6	14	14	14
time for insertion	1c	c	3c	c	c	c	7c	c	c

Total: $c(1+1+3+1+1+1+7+1+1+\dots)$

2 6 14

each number doubles

Expensive operations are followed by cheap operations

$c(1\cdot 2 + 3\cdot 2 + 7\cdot 2)$

$\leq cn + cn = 2cn = O(n)$

Total: $O(n)$

Amortized: $O(1)$

Potential method: provide amortized time

Steps

- define potential function: Φ
  goes from data structure state to a real number

- theorem: Let S_0 be the initial state of the data structure and $S_1, S_2, \dots S_n$ be the state after n operations. Suppose you have a potential function Φ such that

1. $\Phi(S_i) \geq \Phi(S_0)$ potential state never drops after initial state

2. If d_i is the time taken for operation i. then:

$d_i + (\Phi(S_i) - \Phi(S_{i-1})) \leq f(n)$.

Then, the amortized time per operation is $O(f(n))$